PICOLINK
Project Report

Mohd Aasim
Ashmita Subramanian

August 2026

Acknowledgments

We would like to express our **deepest gratitude to our mentors, Omkar Nanajkar and Moksh Panicker**, whose guidance, technical feedback, and continuous support played a crucial role in the development of PicoLink. Their discussions and reviews helped us understand and implement concepts such as embedded graphics, RP2040 peripherals, PIO-based timing, DMA-driven data transfer, USB device classes, and practical PCB development.

We sincerely thank the **SRA-VJTI team** for providing us with the opportunity, resources, and support to work on PicoLink and for fostering an environment that encouraged us to explore, learn, and build.

Mohd Aasim

Ashmita Subramanian

Overview

PicoLink is an embedded graphics and USB project built around the RP2040. The project explores how a low-cost microcontroller can generate a VGA video signal while simultaneously providing USB functionality. The core of the work is the use of the RP2040's Programmable I/O (PIO) blocks and Direct Memory Access (DMA) to generate deterministic video timing and stream pixel data with minimal CPU intervention.

The project began with the study and modification of an existing Pico VGA software stack. The implementation was progressively adapted to support more colour information, while considering the limited SRAM available on the RP2040. The final development direction focused on a practical 16-colour implementation and reliable VGA output.

Alongside VGA generation, PicoLink explores USB composite-device functionality, allowing the RP2040 to act as a USB device with HID and CDC capabilities. This combination makes the platform useful not only as a graphics experiment but also as a compact embedded development and interaction platform.

Table of Content

1.Introduction

1.1 About PicoLink

PicoLink is a project centred on low level embedded graphics and USB device development using the RP2040 microcontroller. Its primary objective is to understand how video timing, **pixel generation**, memory movement, and USB communication can be implemented close to the hardware level.

The project combines hardware and firmware work. On the hardware side, the RP2040 is interfaced with a VGA connector through resistor based RGB outputs. On the firmware side, PIO state machines generate timing-sensitive VGA signals while DMA is used to move pixel data efficiently.

1.2 Problem Statement

Generating a stable VGA signal is timing-sensitive because horizontal synchronization, vertical synchronization, active video intervals, and pixel data must occur at precise times. Performing the complete process directly through software running on the CPU can consume a significant amount of processing time and can make timing less deterministic.

A second challenge is the **limited SRAM of the RP2040**. Increasing colour depth or storing a complete high-resolution framebuffer rapidly increases memory usage. Therefore, PicoLink requires a balance between resolution, colour depth, framebuffer representation, and the available memory.

1.3 Proposed Solution

The proposed solution is to use the RP2040's PIO subsystem as the timing-critical VGA engine. **PIO** state machines generate synchronization signals and output pixel data at a controlled rate. **DMA** feeds the required data to the PIO without requiring the processor to manually write every pixel.

For colour generation, the design uses digital RGB lines connected through resistors to the VGA connector. The implementation was explored at higher colour depths, but the final working direction was reduced to a 4-bit, 16-colour configuration to keep the design practical within the RP2040's memory limitations.

2.Custom PCB Layout

2.1 Introduction

The PicoLink PCB is designed around the **RP2040 microcontroller** and integrates the major hardware required for USB communication, VGA output, external flash storage, power regulation, user input, and hardware expansion. The custom board combines these modules into a single compact platform, providing the required interfaces for the PicoLink system.

2.2 List of Components

RP2040: Main microcontroller responsible for processing and control.

W25Q32JVSS: External QSPI flash memory.

USB Type-C: Provides 5 V power and USB data connection.

AMS1117-3.3: Converts 5 V input to regulated 3.3 V.

VGA Connector: Provides RGB, HSYNC and VSYNC output.

2.3 Circuit Connections

2.3.1 RP2040 Connections

The RP2040 acts as the main controller of the PicoLink PCB. Its GPIO pins are connected to the VGA interface, QSPI flash memory, USB interface, push buttons and expansion headers. The controller is powered from the 3.3 V supply and is provided with the required clock and decoupling circuitry. configuration to keep the design practical within the RP2040's memory limitations.

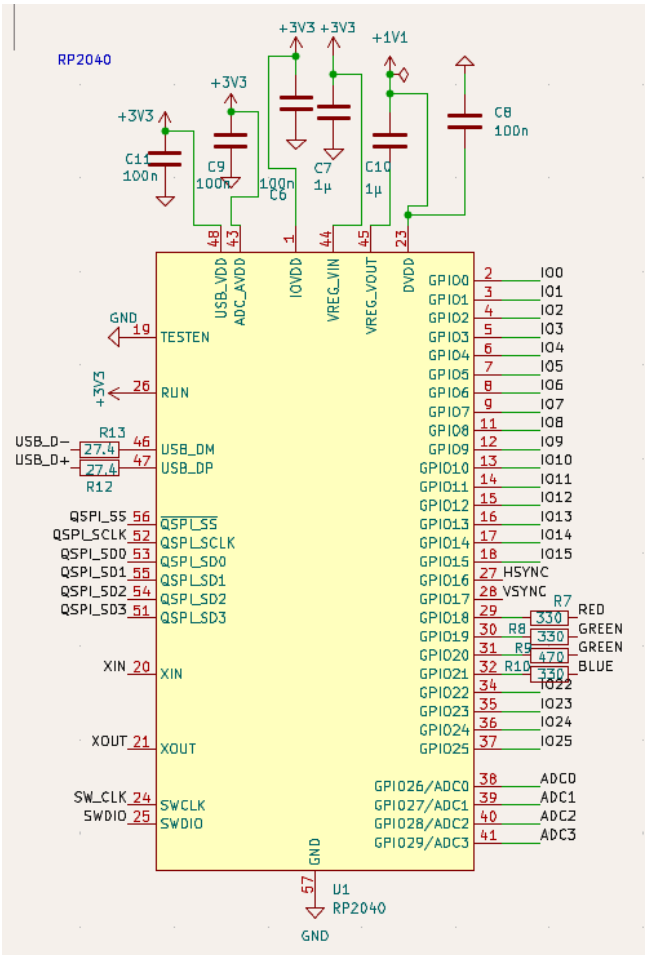

Figure 2.2 RP2040 Schematic

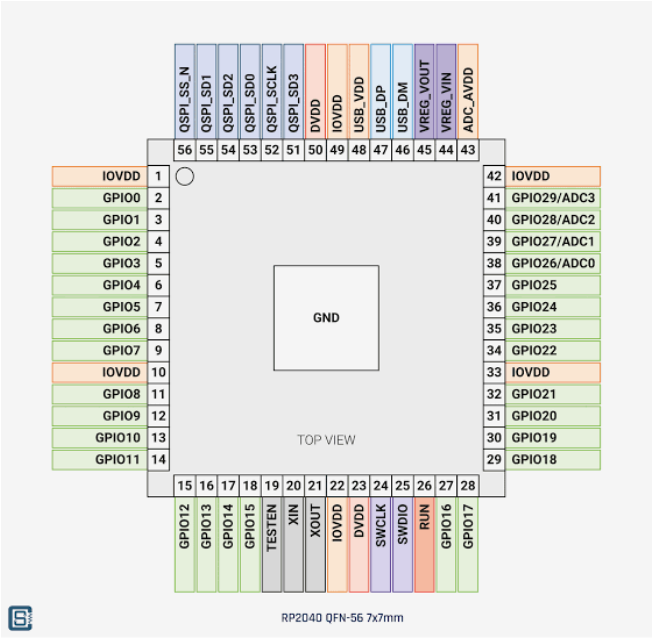

Figure 2.1 RP2040 Pinout Diagram

Connections

2.3.2 USB Connections

The PicoLink PCB provides both **USB Type-C and USB Type-A interfaces**. The USB Type-C connector is used for the primary USB connection and also provides the 5 V input supply to the board. The USB data lines are connected to the RP2040 for USB communication.

The USB Type-A connector provides an additional USB interface for connecting external USB devices. Both connectors are incorporated into the design to provide flexibility for different USB connection and testing requirements.

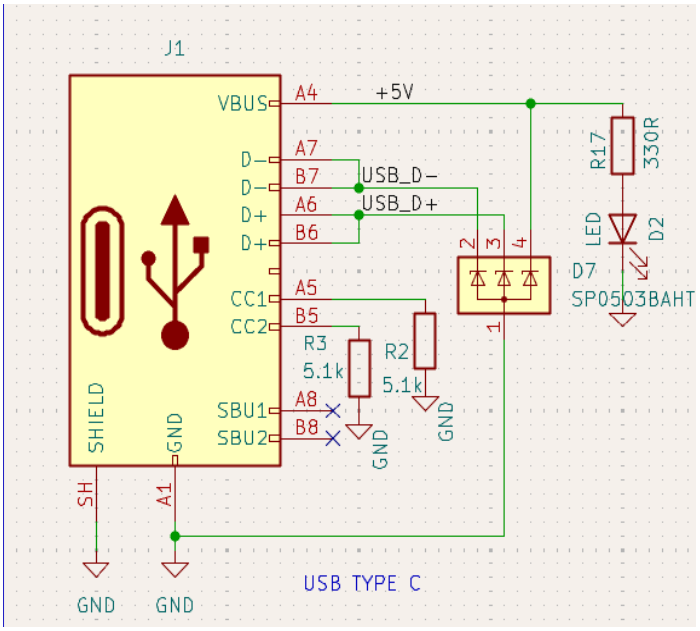

Figure 2.3 USB C Connections

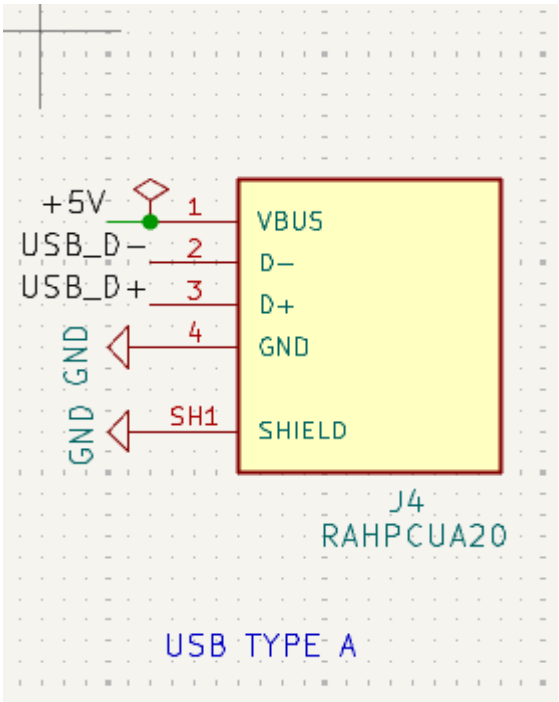

Figure 2.4 USB A Connections

2.3.3 VGA Connections

The PicoLink PCB provides a VGA interface for video output generated by the RP2040. The interface includes separate signals for the **Red, Green, and Blue colour channels**, along with **HSYNC** and **VSYNC** signals for controlling the display timing. The VGA signals are routed from the assigned RP2040 GPIO pins to the VGA connector through the corresponding resistor network. The VGA connector follows the standard **15-pin D-sub (DE-15) pinout**, where pins **1, 2, and 3** are assigned to the Red, Green, and Blue signals respectively, while pins **13 and 14** carry the HSYNC and VSYNC signals. The connector also includes ground connections associated with the RGB and synchronization signals to provide appropriate signal return paths.

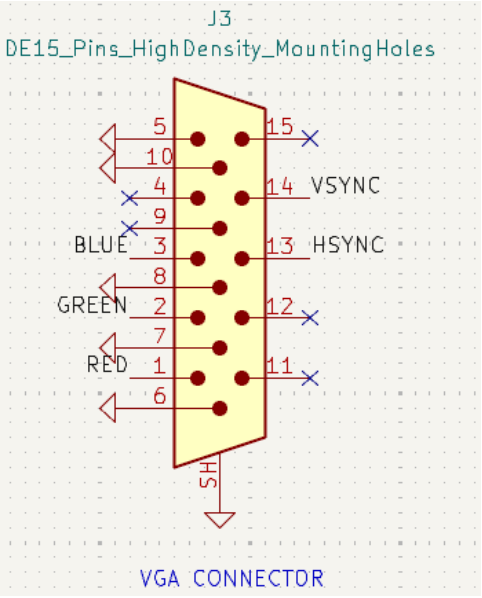

Figure 2.4 VGA Connections

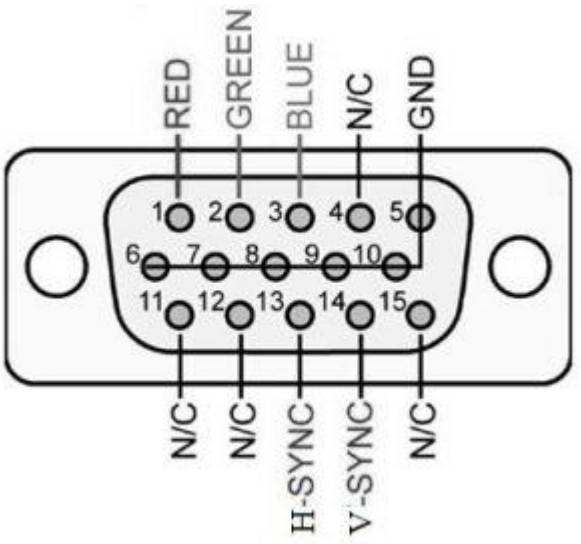

Figure 2.5 VGA Pin Layout

2.3.4 External Flash Memory

The **W25Q32JVSS** flash memory is connected to the RP2040 using the QSPI interface. The memory uses dedicated clock, chip-select and data signals and operates from the 3.3 V supply.

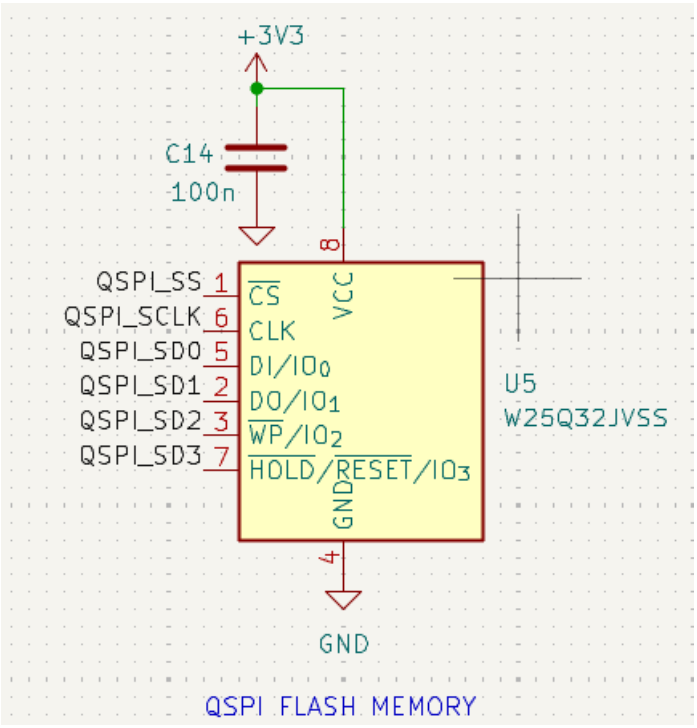

Figure 2.6 QSPI flash memory connections

2.3.5 Power Supply

The PCB receives 5 V through the USB interface. **The AMS1117-3.3** regulator converts this input into a regulated 3.3 V supply for the RP2040 and other components. Capacitors are provided around the regulator for stable operation, along with an LED to indicate the presence of the supply.

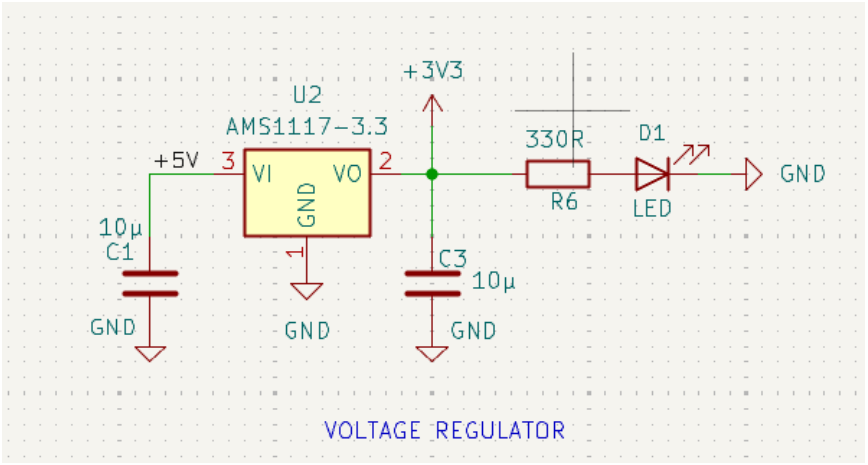

Figure 2.7 3.3 V voltage regulator circuit

2.3.5 Clock and Push Buttons

A crystal is connected to the **XIN** and **XOUT** pins of the RP2040 to provide the required clock reference. The PCB also contains push buttons connected to GPIO pins, which can be used for control and testing purposes.

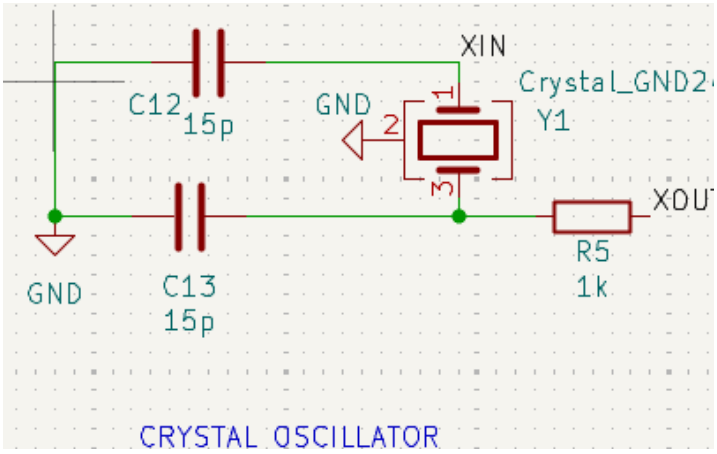

Figure 2.8 Crystal oscillator circuit

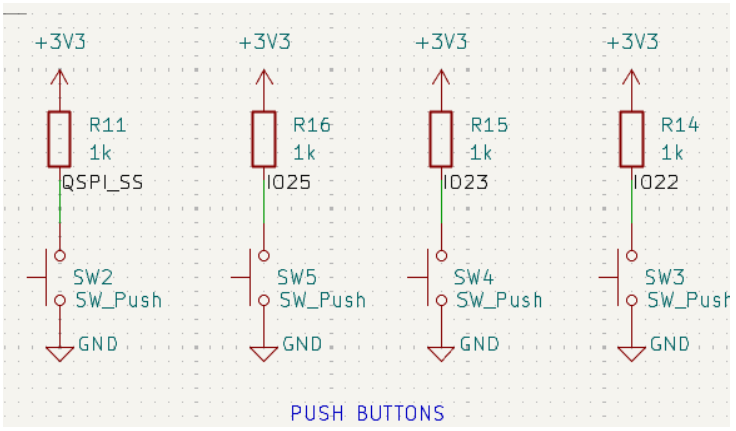

Figure 2.9 Push Buttons

3.VGA Generation

3.1 Introduction to VGA

VGA generation is one of the main functions implemented in the PicoLink project. The RP2040 is used to generate the video signals required by a VGA display, including the RGB colour signals and the horizontal and vertical synchronization signals. Instead of using a dedicated VGA controller, the project makes use of the RP2040's programmable hardware resources to generate the required timing and pixel data.

The VGA output is generated by coordinating the pixel data with the synchronization signals.

3.2 VGA Signal Components

A VGA video signal consists of three colour channels and two synchronization signals. The three colour channels are **Red, Green, and Blue (RGB)**, which determine the colour of each pixel. The two timing signals are **HSYNC** and **VSYNC**.

HSYNC is associated with the timing of each horizontal line of the display, while **VSYNC** controls the transition between successive frames. The RGB signals are generated during the active display region, while the synchronization signals maintain the timing required by the monitor.

For PicoLink, these signals are generated using dedicated RP2040 GPIO connections and are routed to the VGA connector on the custom PCB.

3.3 VGA Timing

A VGA display does not continuously interpret the RGB signals as visible pixels. Each horizontal line follows a defined timing sequence consisting of an active video region and timing intervals associated with synchronization.

Similarly, a complete frame consists of multiple horizontal lines followed by the vertical synchronization sequence. Therefore, the VGA generator has to maintain accurate timing for both the horizontal and vertical directions.

The PicoLink implementation generates these timing signals continuously so that the display can determine the current position within the frame and correctly associate the generated pixel data with its position on the screen.

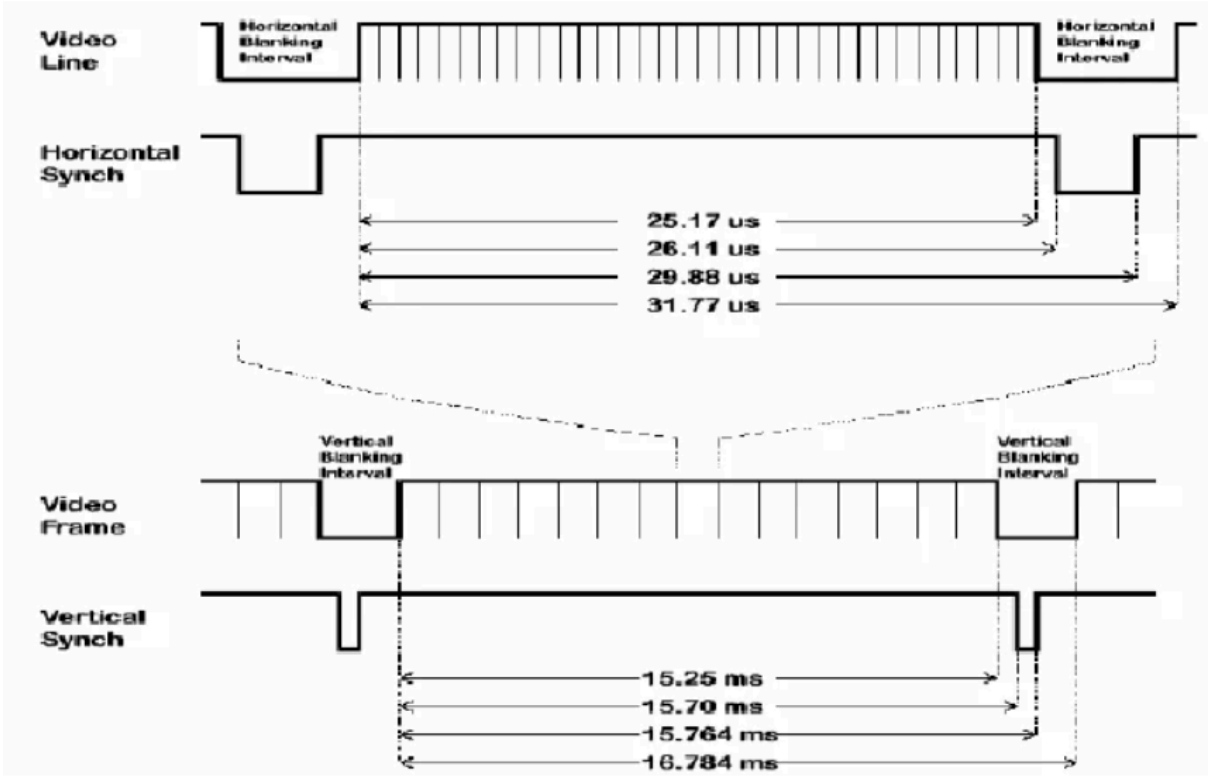

Figure 3.1 VGA Timing Diagram

3.4 PIO-Based Signal Generation

The RP2040 includes a Programmable I/O (PIO) subsystem that can execute small dedicated programs independently of the main processor. This makes PIO suitable for applications where signals have to be generated with predictable timing.

In PicoLink, PIO is used as part of the VGA generation process. Instead of making the main CPU manually control every timing transition, the PIO state machines handle time-critical signal generation. This allows the processor to perform other tasks while the video hardware maintains the required timing.

The PIO program controls the sequence in which the synchronization and pixel-related signals are produced. The timing of these operations is determined by the PIO instructions and its clock configuration.

3.5 Pixel Data and Framework

To display graphics, the VGA generator requires pixel information corresponding to the current position on the screen. PicoLink uses a framebuffer-based approach in which pixel data is stored in memory and read during video generation.

Each location in the framebuffer represents the colour information of a corresponding pixel. During display generation, the stored pixel information is converted into the appropriate RGB output signals.

The framebuffer size depends on both the display resolution and the number of bits used to represent each pixel. Increasing either the resolution or colour depth increases the required memory. This became an important consideration during PicoLink development because the available RAM of the RP2040 places a practical limit on the size of a directly stored framebuffer.

3.6 Color Generation

The RGB output is generated by combining the digital signals assigned to the red, green and blue channels. Different combinations of logic levels produce different colour values at the VGA interface.

The number of available colours depends on the number of bits allocated to the RGB channels. Increasing the colour depth provides more possible colours but also increases the amount of memory required when a framebuffer is used.

During PicoLink development, colour depth and display resolution were therefore considered together because both directly affect the memory requirement of the framebuffer.

3.7 VGA Signal Verification

After implementing the VGA generation system, the generated synchronization signals were tested using an oscilloscope. HSYNC and VSYNC were observed directly to verify that the RP2040 was producing the required periodic signals.

The oscilloscope measurements provide a hardware-level verification of the signal generation rather than relying only on the final monitor output. The VGA output was subsequently tested with a display to verify that the generated synchronization and colour signals were correctly interpreted.

Figure 3.2 Hysnc Signal Verification on DSO

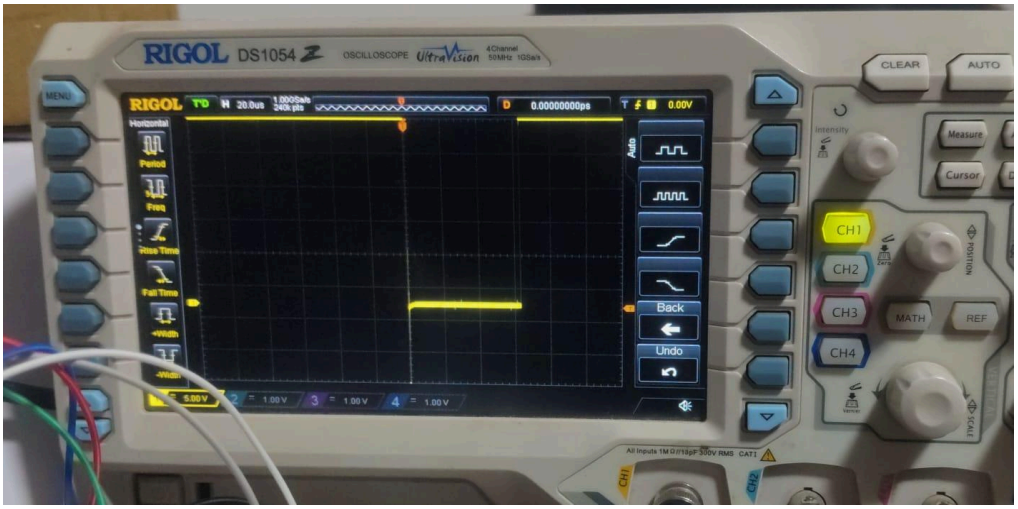

Figure 3.3 Vsync Signal Verification on DSO

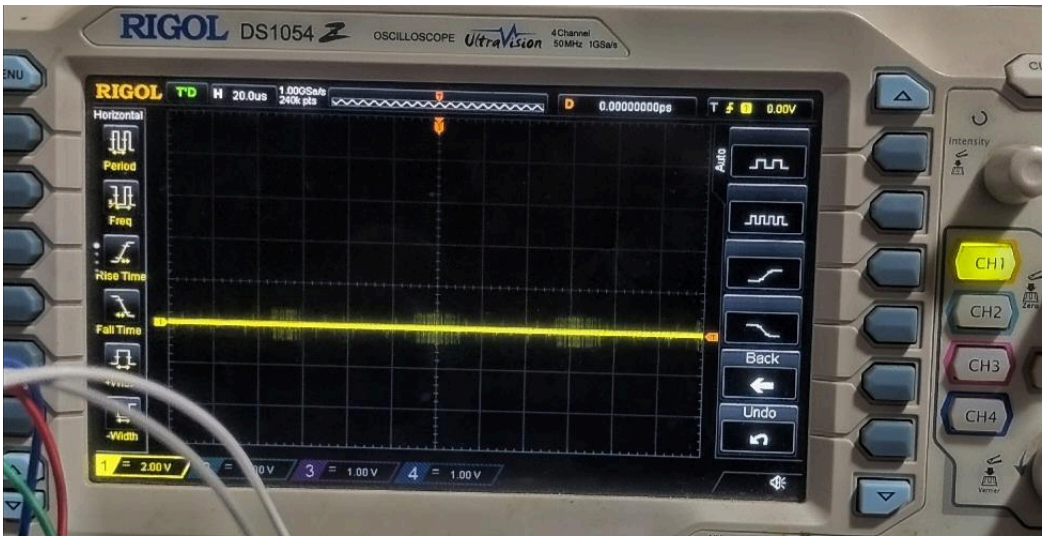

Figure 3.3 RGB Signal Verification on DSO

4. Graphics and Framebuffer

4.1 Introduction

The graphics system of PicoLink is responsible for storing and displaying pixel information on the VGA output. The RP2040 generates the required VGA timing signals while the graphics data is stored in memory and supplied to the video-generation pipeline.

A framebuffer is used to represent the image that is displayed on the screen. Each location in the framebuffer corresponds to a pixel, and the value stored at that location determines the colour that is produced on the VGA output.

The implementation of the framebuffer required careful consideration of the available memory of the RP2040, since the amount of memory required increases with both display resolution and colour depth.

4.2 Frame Buffer

The framebuffer is a region of memory that stores the pixel data for a complete display image. Instead of generating the graphics directly at the VGA output, the required image can first be constructed in the framebuffer and then read during the display process.

For a framebuffer with width (W), height (H), and (B) bits per pixel, the required memory can be calculated as:

$$\text{Framebuffer Memory} = W \times H \times 8B$$

$$\text{Framebuffer Memory} = 640 \times 480 \times 84 = 307200 \times 0.5$$

$$= 153600 \text{ bytes}$$

This shows why framebuffer size becomes an important limitation on a microcontroller with limited internal RAM.

4.3 Pixel Representation

Each pixel in the framebuffer is represented using a fixed number of bits. These bits determine the colour information that is eventually sent to the RGB outputs.

In the PicoLink implementation, a **4-bit colour representation** was used, allowing:

2^4 = 16 different colour values.

The pixel data is therefore represented using a compact format rather than storing a complete multi-byte colour value for every pixel. This reduces the memory required by the framebuffer while still providing multiple colour combinations for the VGA output.

4.4 Resolution and Memory Requirement

The framebuffer memory requirement is directly dependent on the selected display resolution. Increasing the horizontal or vertical resolution increases the total number of pixels that have to be stored.

Resolution	Pixels
320 x 240	76,800
640 x 480	307,200

At the same colour depth, the 640 × 480 framebuffer requires four times as many pixel locations as a 320 × 240 framebuffer.

Similarly, increasing the number of bits used for each pixel increases the amount of memory required per pixel. Therefore, resolution and colour depth have to be selected together when designing a framebuffer-based graphics system for a resource-constrained microcontroller.

4.5 Graphics Rendering

Graphics are generated by modifying the values stored in the framebuffer. A graphics operation can change one or more pixel locations, after which the VGA generation system reads the updated pixel information during the corresponding display period.

This provides a separation between the graphics generation and the physical VGA signal generation. The application determines what should appear on the screen, while the video-generation system is responsible for transmitting the stored pixel information with the appropriate timing.

This approach also makes it possible to implement different graphics operations such as drawing pixels, lines, shapes, text, and other visual elements using the same framebuffer.

Figure 4.1 Graphics Displayed using PicoLink framebuffer

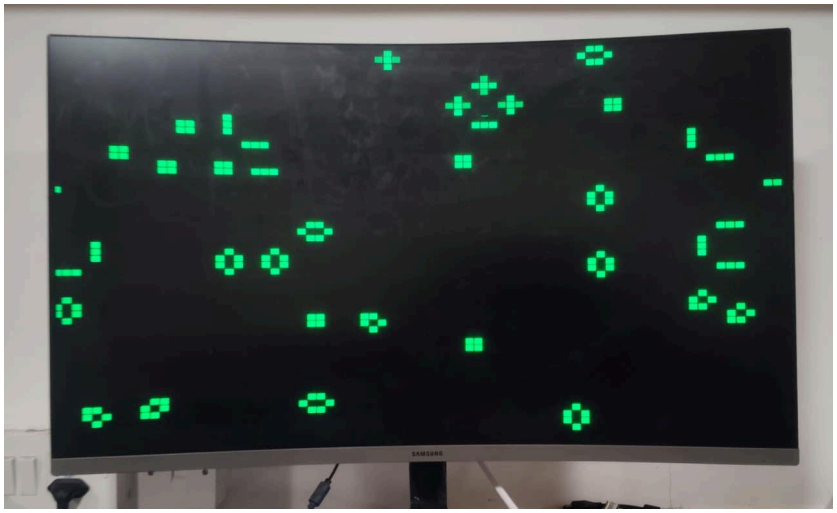

Figure 4.2 : Graphics Displayed using PicoLink framebuffer

4.6 Memory Limitations During Development

One of the major challenges encountered during the PicoLink graphics implementation was the limited RAM available on the RP2040.

An initial implementation attempted to use a **640 × 480 framebuffer with higher colour information**. The resulting memory requirement became too large for the available RAM, leading to memory and linker limitations during compilation.

This demonstrated an important trade-off in the design: increasing resolution and colour depth improves the visual output but simultaneously increases memory consumption.

The implementation was therefore modified to use a more memory-efficient pixel representation. The final implementation used **4-bit colour, providing 16 colours**, while retaining the required VGA output functionality.

5. USB Implementation

5.1 Introduction

USB communication is another important part of the PicoLink project. The RP2040 provides a built-in USB interface that can be used to communicate with a host computer and to implement different USB device classes.

In PicoLink, the USB interface is used to provide communication between the microcontroller and the host system. The implementation combines multiple USB functions into a single device, allowing the board to perform different types of communication through one USB connection.

5.2 RP2040 USB Interface

The RP2040 contains an integrated USB 1.1 controller and PHY, allowing USB communication without requiring a separate USB-to-UART converter for the main USB interface.

The USB D+ and D− signals from the USB connector are connected directly to the USB interface of the RP2040. The USB controller handles the low-level communication with the host, while the firmware defines how the device behaves once it is connected.

5.3 USB HID

Human Interface Device (HID) is a USB device class commonly used for peripherals such as keyboards, mice and controllers. HID devices can communicate with the host using standardized USB descriptors and reports.

In PicoLink, the HID interface can be used to make the RP2040 appear to the host computer as a USB input device. The firmware controls the data contained in the HID reports and determines how the host interprets the received information.

This provides a way for PicoLink to interact with the host without requiring a dedicated application-specific USB driver.

5.4 USB CDC

The Communication Device Class (CDC) provides a standardized method for implementing serial-style communication over USB. A CDC interface allows the host computer to communicate with the RP2040 through a virtual serial connection.

In PicoLink, the CDC interface can be used for communication between the firmware and the host computer. It is particularly useful during development because messages and debugging information can be exchanged without requiring a separate physical serial connection.

5.5 Composite USB Device

PicoLink combines the HID and CDC interfaces into a **composite USB device**. This allows a single USB connection to expose more than one USB function to the host.

After connecting PicoLink to a computer, the host can identify the different interfaces provided by the device. The HID interface handles input-related communication, while the CDC interface provides serial-style communication.

The composite approach allows multiple functions to coexist without requiring separate USB connectors for each interface.

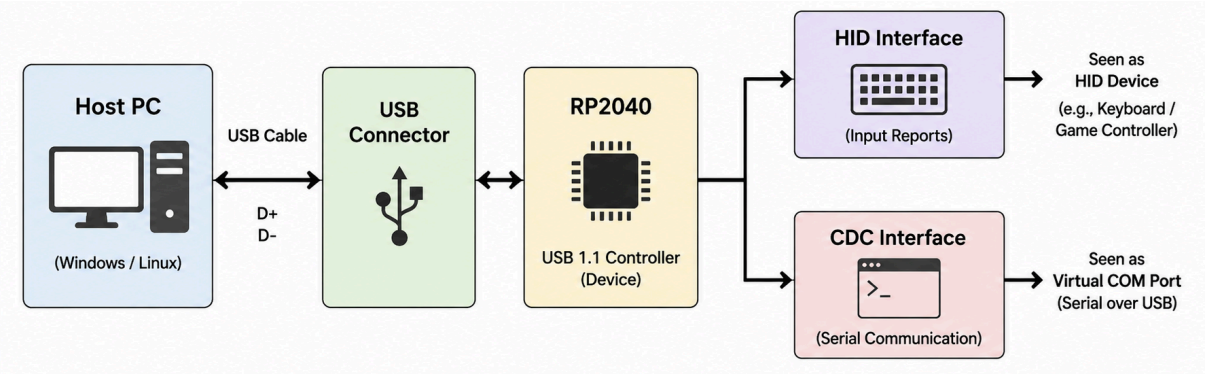

Figure 5.1 : Block Diagram of USB Composite Device

5.6 Final USB Composite Implementation

The Final USB implementation of PicoLink combines **HID and CDC functionality within a single composite USB device**. Both interfaces are implemented through the RP2040's USB peripheral and are accessible through a single physical USB connection to the host computer.

The **HID interface** provides the device-side functionality required for HID-based communication, while the **CDC interface** provides a virtual serial communication channel between the PicoLink board and the host system. The composite configuration allows both interfaces to operate simultaneously without requiring separate USB connections.

The final implementation was integrated into the PicoLink firmware and verified on the host computer. The resulting device exposes both HID and CDC interfaces, demonstrating the successful integration of multiple USB device classes into the PicoLink system.

.

Figure 5.2 : USB Composite Interface — HID and CDC

6.Development Process

The development of PicoLink was carried out over a period of two months, with the work divided into four biweekly phases. Each phase focused on a specific set of hardware and software tasks, gradually taking the project from initial learning and PCB design to VGA implementation, game development, USB integration, and final PCB assembly.

6.1 Week 1–2 — Resource Study and Initial PCB Design

During the first two weeks, the focus was on building the technical foundation required for the PicoLink project. Resources related to **C programming, FreeRTOS, and PCB design** were studied to understand the software and hardware concepts required for the project.

Alongside this learning phase, the requirements of the PicoLink hardware were analyzed and an initial **RP2040-based PCB design** was developed. The schematic was prepared by selecting the required components and planning their connections according to the intended functionality of the board.

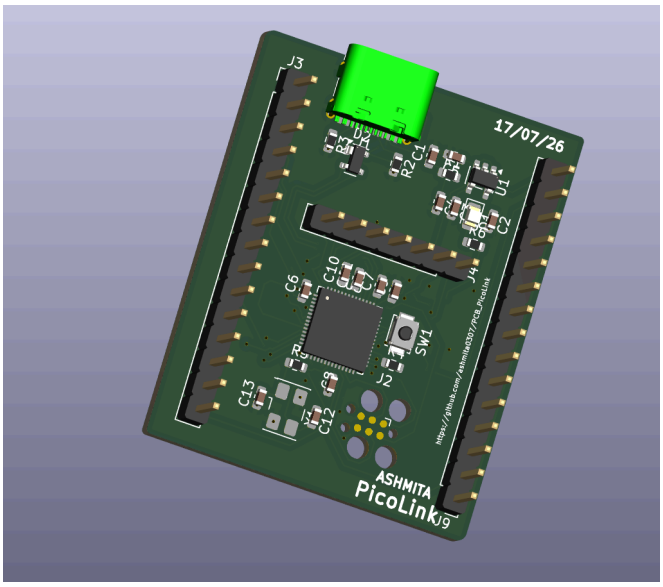

Figure 6.1 PCB Design-(a)

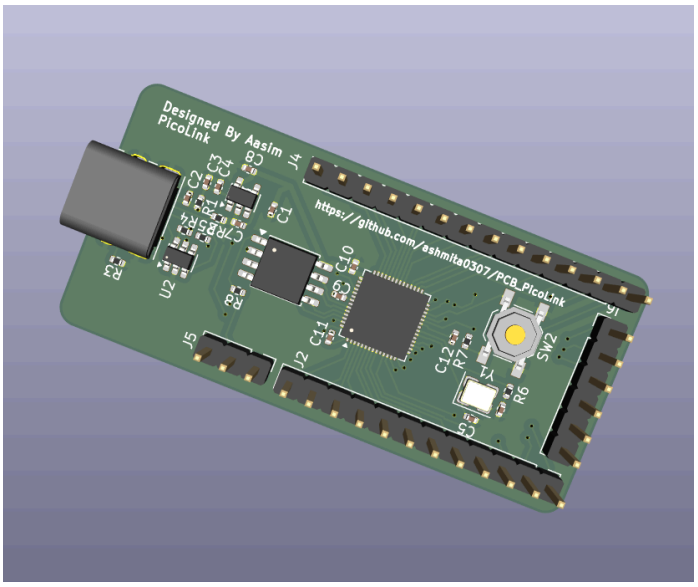

Figure 6.2 RP2040 PCB-(b)

6.2 Week 3–4 — Hardware Connections and VGA Library Implementation

During the third and fourth weeks, the required RP2040 hardware connections were implemented on a **breadboard** as an initial prototype. The connections between the RP2040 and the VGA interface were established according to the planned design, allowing the circuit to be tested before moving to the custom PCB.

The VGA library was then implemented to generate video output from the RP2040. The existing library was studied to understand its structure and the method used for VGA signal generation. It was then adapted to work with the PicoLink hardware and its assigned GPIO connections.

This phase resulted in the **first working VGA output on the breadboard**, providing the foundation for further development of the graphics system and subsequent PCB implementation.

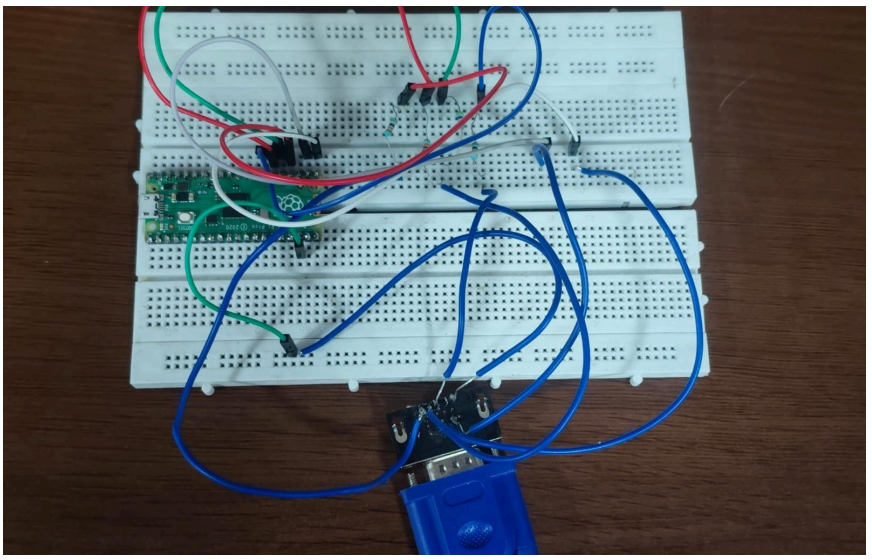

Figure 6.3 : PicoLink VGA Breadboard Connections

6.3 Week 5–6 — VGA Development and PCB Completion

During the fifth and sixth weeks, the focus shifted towards improving the VGA implementation and developing an application using the graphics system. Another VGA library was studied and implemented to explore an alternative approach to VGA generation. Along with this, the firmware for the PicoLink system was developed to control the implemented hardware and video output.

After establishing the graphics functionality, a **Space Invaders game** was developed as an application to demonstrate the VGA graphics capabilities of PicoLink. This provided a practical way to verify the graphics rendering and interaction between the firmware and the VGA output.

In parallel, the **PicoLink PCB design was completed in KiCad**, including the required component placement, connections, and routing for the final board.

Figure 6.4 : Space invader’s Implementation

Figure 6.5 : Final PicoLink PCB Design

6.4 Week 7–8 — USB Composite, Game Development and PCB Assembly

During the seventh and eighth weeks, the focus was on completing the remaining PicoLink features and moving towards the final hardware implementation. The USB functionality was extended by implementing **HID and CDC as a composite USB device**, allowing both interfaces to operate through the same USB connection.

The graphics system was further demonstrated by developing a second game, **Pac-Man**, using the PicoLink VGA output and graphics capabilities. This provided another practical application of the implemented graphics system.

Alongside the software development, the completed PCB was assembled and the hardware implementation was documented. The assembly process included populating the board with the required components and preparing the assembled PCB for subsequent hardware testing and verification.

Figure 6.5 : Pacman Implementation

Figure 6.6 : Final PicoLink PCB

7. Future Scope

The PicoLink platform can be further developed by improving its graphics capabilities, USB functionality, and overall hardware design. The modular nature of the RP2040-based system provides scope for extending the existing implementation and exploring more advanced applications.

7.1 Higher Resolution and Colour Depth

The graphics system can be optimized to support higher display resolutions and greater colour depth. More memory-efficient rendering techniques, such as scanline-based rendering, could reduce framebuffer memory requirements and allow better visual output within the RP2040's memory constraint

7.2 Advanced Graphics

The existing graphics system can be extended with additional graphics primitives, improved text rendering, sprites, animations, and more complex graphical applications. This could allow PicoLink to support a wider range of games and interactive applications.

7.3 More Applications and Games

The PicoLink graphics platform can be used to develop additional games and interactive applications beyond **Space Invaders and Pac-Man**. This would provide further opportunities to evaluate the performance and capabilities of the graphics syst

7.4 Optimized Video Pipeline

The VGA pipeline can be further optimized by improving the interaction between **PIO, DMA, and framebuffer memory**. Better utilization of these hardware resources could reduce CPU overhead and provide smoother graphics generation.

8. Conclusion

The PicoLink project provided practical experience in designing and developing an embedded graphics system using the **RP2040**. Over the course of the project, VGA generation was implemented by combining **PIO-based timing, DMA-driven pixel transfer, and framebuffer-based graphics**, allowing the system to generate a functional VGA output.

The project also involved developing a practical **4-bit colour implementation with 16 colours**, taking into consideration the limited SRAM available on the RP2040. The graphics system was further demonstrated through applications such as **Space Invaders and Pac-Man**, providing a practical demonstration of the implemented VGA and graphics functionality.

Alongside the graphics system, **USB HID and CDC were integrated as a composite USB device**, allowing multiple USB interfaces to operate through a single connection. The hardware development progressed from an initial breadboard prototype to the design, completion, and assembly of a custom PicoLink PCB.

Overall, PicoLink provided hands-on experience in **embedded C programming, RP2040 peripherals, PIO, DMA, VGA signal generation, framebuffer management, USB device development, PCB design, and hardware debugging**. The project also highlighted the importance of balancing system performance with the hardware resources available in an embedded platform.

9. References

1. Raspberry Pi Ltd., *RP2040 Datasheet*, Raspberry Pi, 2024. RP2040 Datasheet
2. razorArnov, *PICO-VGA-BOARD — vga_graphics.cpp*, GitHub. The file was used as a reference for the VGA graphics implementation. PICO-VGA-BOARD — vga_graphics.cpp
3. *C Programming Series*, YouTube playlist. Used during the initial study of C programming. C Programming Series
4. *Introduction to RTOS*, YouTube playlist. Used for studying RTOS concepts during the initial learning phase. Introduction to RTOS
5. *VGA/RP2040 Related Tutorial*, YouTube. Used as a reference during the development and implementation of the VGA system. VGA/RP2040 Tutorial